Session 01 - Your First Encrypted Message - Tests

Python Contoso Vault - Session 01 of 07

Part 1 - Core: shift.py

Goal

Read a message and a key. Encrypt the message, then decrypt it again to prove the round trip works. Keep the case of each letter. Leave every character that is not a letter unchanged.

Run `python shift.py` in the terminal. Type the message and press Enter, then type the key and press Enter.

Instructions

1. Read a message as a string.
2. Read a key from 0 to 25 as an integer.
3. Shift each letter of the message forward by the key, within its own case.
4. Wrap any shifted letter that passes the end of its alphabet.
5. Leave every character that is not a letter unchanged.
6. Print the ciphertext.
7. Shift each letter of the ciphertext back by the key.
8. Wrap any shifted letter that passes the start of its alphabet.
9. Print the plaintext.
10. Prove that the plaintext matches the message.

Constraints

- Read each value with a prompt on the same line, as in `input("Message: ")`. The prompts print in this order: `"Message: "` for the message, then `"Key: "` for the key. Each prompt is its label, a colon and one trailing space. Type each prompt exactly as shown. A program with different prompt text does not meet the test plan.
- After the two prompts, the output is exactly two lines: `Ciphertext: <text>`, then `Plaintext: <text>`. Put one space after each colon.

Design

The following rules say how to write the code. Follow them.

- Wrap shifted letters without `%`.
- Prove the round trip with an `assert`.
- Give every variable a type hint.

Input validation

Not required. Assume the key is always a whole number from 0 to 25. Any message is valid, including an empty one.

Test plan

The program prints both prompts first, so the Expected output cells show the two lines printed after them.

Test	Message	Key	Expected output	Actual output	Pass
1	`Meet at Contoso HQ!`	3	`Ciphertext: Phhw dw Frqwrvr KT!` `Plaintext: Meet at Contoso HQ!`		
2	`Zebra zone 42, room 7.`	25	`Ciphertext: Ydaqz ynmd 42, qnnl 7.` `Plaintext: Zebra zone 42, room 7.`		
3	`Contoso HQ`	0	`Ciphertext: Contoso HQ` `Plaintext: Contoso HQ`		
4	`Contoso HQ`	1	`Ciphertext: Dpouptp IR` `Plaintext: Contoso HQ`		
5	`Contoso HQ`	13	`Ciphertext: Pbagbfb UD` `Plaintext: Contoso HQ`		
6	`Contoso HQ`	25	`Ciphertext: Bnmsnrn GP` `Plaintext: Contoso HQ`		
7	`2026-11-01, 9:00!`	7	`Ciphertext: 2026-11-01, 9:00!` `Plaintext: 2026-11-01, 9:00!`		
8	`mIxEd CaSe`	4	`Ciphertext: qMbIh GeWi` `Plaintext: mIxEd CaSe`		
9	empty line	5	`"Ciphertext: "` `"Plaintext: "`		

Done criteria

- [ ] Every test plan row prints exactly the expected lines.
- [ ] The script runs from the terminal with `python shift.py`.

Part 2 - Challenge: crack_caesar.py

Goal

Read a ciphertext. Find the key that reveals the word `"contoso"`, in capital letters, small letters or a mix of both.

Run `python crack_caesar.py` in the terminal. Type the ciphertext and press Enter.

Instructions

1. Read a ciphertext as a string.
2. Try each key from 0 to 25, as an integer, in turn.
3. Decrypt the ciphertext with the current key, as in `shift.py`.
4. Stop at the first plaintext that contains `"contoso"` in any case.
5. Print the key.
6. Print the plaintext.
7. Report that no key was found if no key works.

Constraints

- Read the ciphertext with a prompt on the same line, as in `input("Ciphertext: ")`. The prompt is `"Ciphertext: "`, its label, a colon and one trailing space. Type each prompt exactly as shown. A program with different prompt text does not meet the test plan.
- After the prompt, a found key prints exactly two lines, `Key: <n>` then `Plaintext: <text>`. A miss prints exactly one line, `No key found`.
- The printed key is the one that turned the plaintext into the ciphertext.

Design

The following rules say how to write the code. Follow them.

- Wrap shifted letters without `%`.
- Decrypt with the same logic as `shift.py`.
- Give every variable a type hint.

Input validation

Not required. Any ciphertext is valid, including an empty line.

Test plan

The program prints the prompt first, so the Expected output cells show the lines printed after it.

Test	Ciphertext	Expected output	Actual output	Pass
1	`FRQWRVR PHHWV DW QRRQ`	`Key: 3` `Plaintext: CONTOSO MEETS AT NOON`		
2	`Sbujo ha uvvu`	`No key found`		
3	`CONTOSO PAYROLL`	`Key: 0` `Plaintext: CONTOSO PAYROLL`		
4	`BNMSNRN OZXQNKK`	`Key: 25` `Plaintext: CONTOSO PAYROLL`		
5	`iutzuyu vgexurr oy jak`	`Key: 6` `Plaintext: contoso payroll is due`		
6	`Lxgw max vhgMHLh unwzxm`	`Key: 19` `Plaintext: Send the conTOSo budget`		
7	empty line	`No key found`		

Done criteria

- [ ] Every test plan row prints exactly the expected lines.
- [ ] The decrypt gives the same output as `shift.py` on the same key.

Marking

This take-home is marked out of 100, with a pass mark of 60.

Part	Tests	Design	Total
Core	64	16	80
Challenge	16	4	20
Total	**80**	**20**	**100**

Guidelines

- Test marks show that the program works. The instructor's tests run it on every row of the test plan, plus further cases.
- Design marks show that the code is built well. The instructor reads the code against each Design rule.
- A pass needs 60 marks from the Core alone. Challenge marks count towards the total once the Core reaches 60.
- Marking uses the last commit pushed by 11:59 PM on the day before the next session.
- The submission is one GitHub link to the fork, posted on the portal.